

Step 3 - Design deep dive

In the high-level design, the entire system is broken down into two parts:

- Video uploading flow
- Video streaming flow

In this section, we refine both flows with important optimizations and introduce error handling mechanisms.

Video transcoding

When you record a video, the device (usually a phone or camera) gives the video file a certain format. If you want the video to be played smoothly on other devices, the video must be encoded into compatible bitrates and formats.

Bitrate is the rate at which bits are processed over time.

- Higher bitrate → better quality
- Higher bitrate → more bandwidth and processing power required

Why video transcoding is important

1. Reduce storage consumption

Raw videos consume huge amounts of storage.

Example:

- 1-hour HD video at 60 FPS may take hundreds of GB.

Thus, videos must be compressed efficiently.

2. Device and browser compatibility

Different devices support different video formats.

Examples:

- Mobile devices
- Smart TVs
- Browsers

Therefore, videos are converted into multiple formats.

3. Adaptive streaming

Users with:

- High bandwidth receive high-quality videos
- Low bandwidth receive lower-quality videos

This improves playback smoothness.

4. Dynamic network adaptation

Internet speed changes frequently, especially on mobile devices.

The platform automatically changes:

- Resolution
- Bitrate
- Codec

to avoid buffering.

Video encoding formats

Video encoding formats generally contain two parts:

1. Container

A container stores:

- Video stream

- Audio stream
- Metadata

Examples:

- .mp4
- .avi
- .mov

2. Codec

Codec compresses and decompresses video.

Goals:

- Reduce file size
- Preserve video quality

Popular codecs:

- H.264
- VP9
- HEVC

Directed Acyclic Graph (DAG) model

Transcoding a video is computationally expensive and time-consuming. Different content creators may also require different processing pipelines.

Examples:

- Watermarks
- Thumbnails
- Multiple resolutions
- Different codecs

To support flexible workflows and maintain high parallelism, a **Directed Acyclic Graph (DAG)** programming model is used.

In a DAG:

- Nodes represent tasks
- Edges represent dependencies

Tasks can run:

- Sequentially
- Parallely

This improves flexibility and scalability

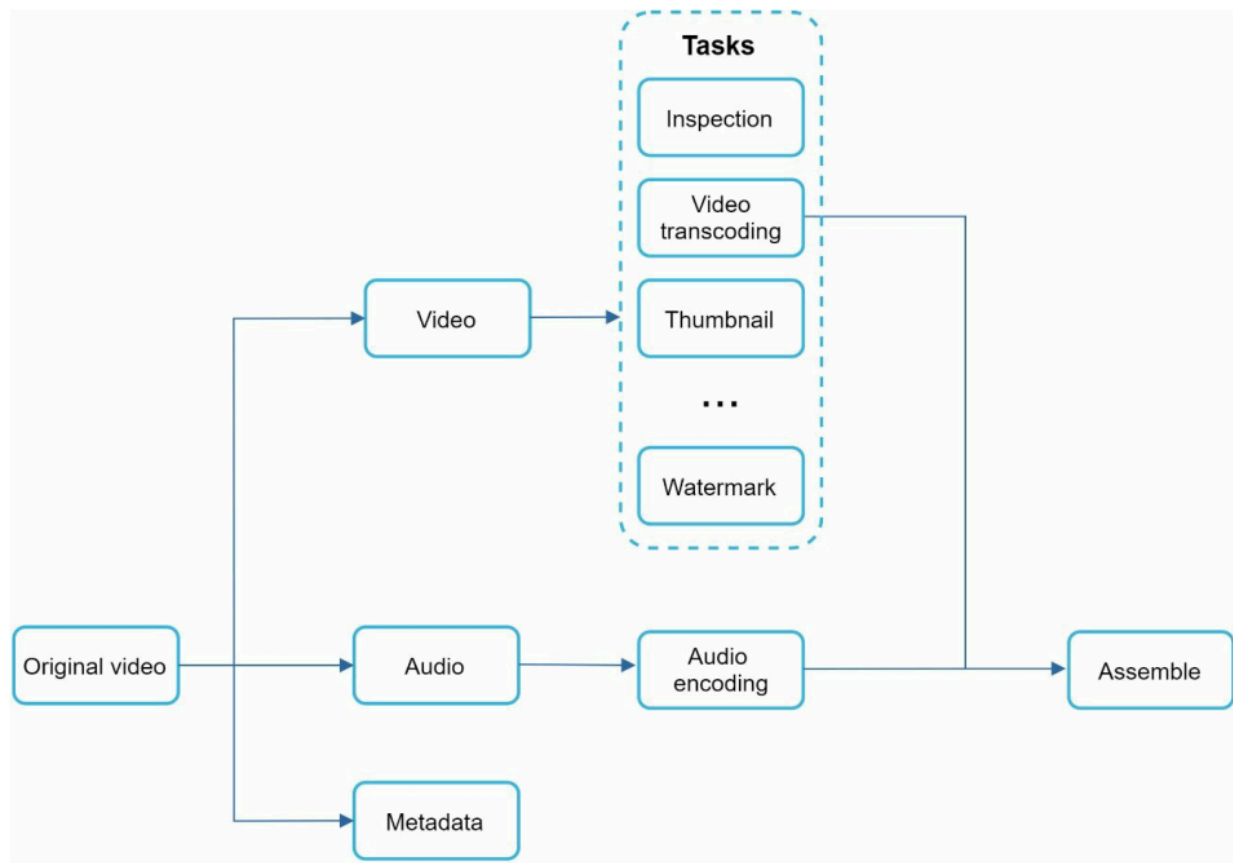


Figure shows the DAG model for video transcoding.

DAG-based transcoding flow

In **Figure**, the original video is divided into:

- Video stream
- Audio stream
- Metadata

Different processing tasks are then applied.

Inspection

Checks:

- Video quality
- Corrupted files
- Unsupported formats

Video encoding

Videos are encoded into:

- Different resolutions
- Different codecs
- Different bitrates

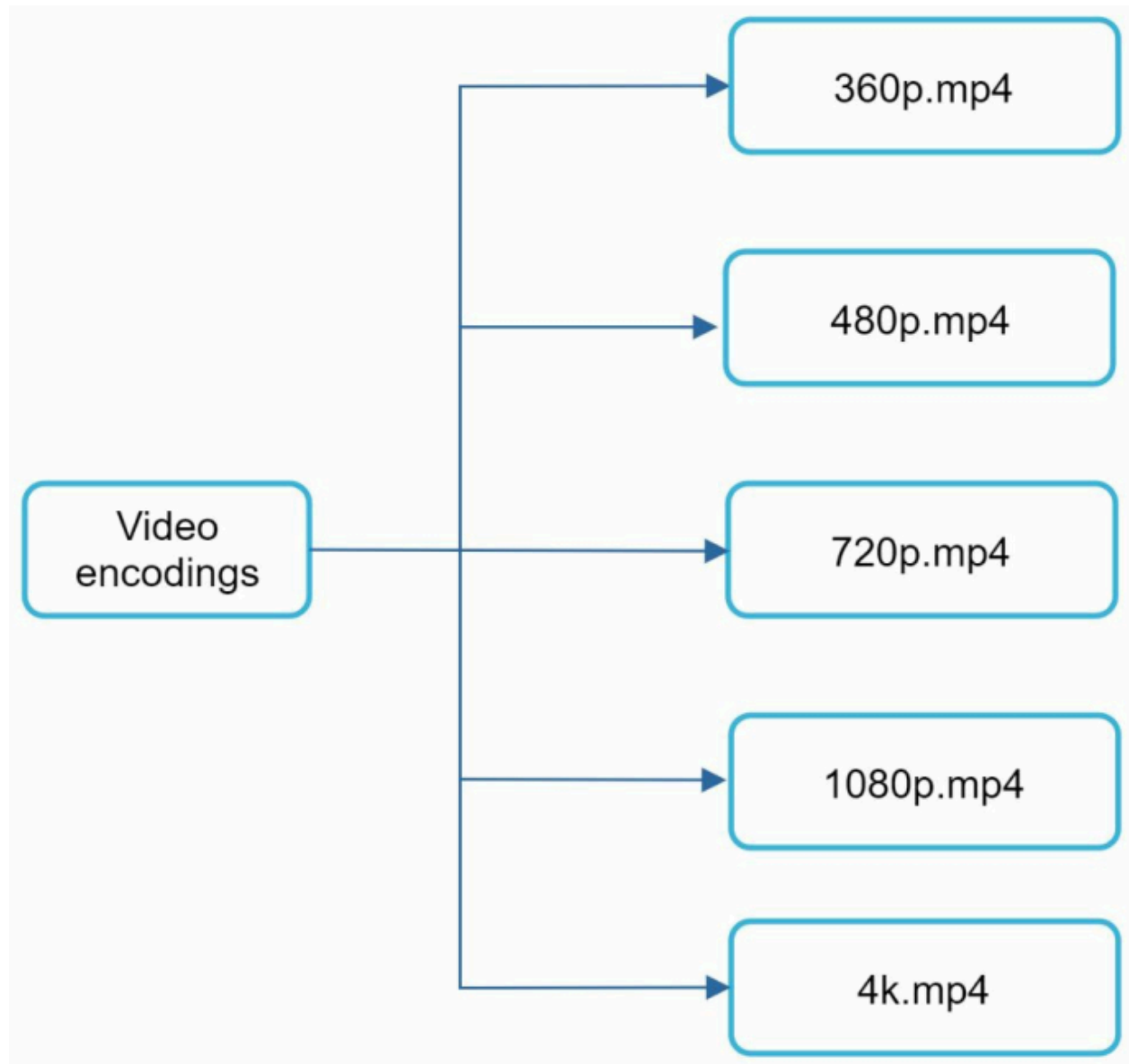


Figure shows examples of encoded video outputs.

Examples:

- 240p
- 480p
- 720p
- 1080p

Thumbnail generation

Thumbnails may be:

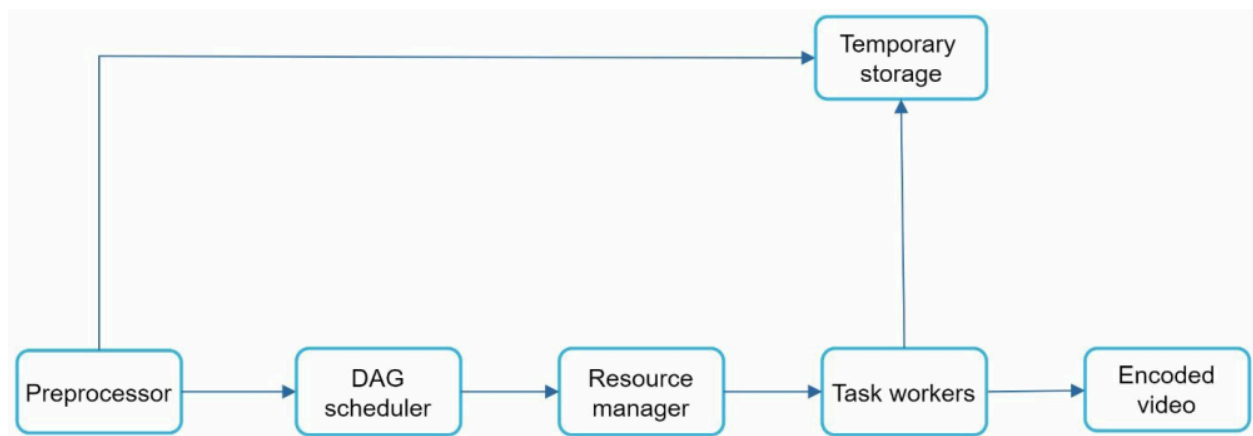
- Uploaded by users
- Automatically generated by the system

Watermarking

Watermarks add:

- Branding
- Copyright information
- Ownership details

Video transcoding architecture



The proposed transcoding architecture is shown in **Figure**.

The architecture contains six major components:

1. Preprocessor
2. DAG Scheduler
3. Resource Manager
4. Task Workers

5. Temporary Storage
6. Encoded Video Output

1. Preprocessor

The preprocessor has four major responsibilities.

a) Video splitting

The uploaded video is split into smaller chunks called **GOPs (Group of Pictures)**.

A GOP:

- Is an independently playable unit
- Usually contains a few seconds of video

Benefits:

- Easier parallel processing
- Faster transcoding
- Better fault recovery

b) Support old devices

Some old devices or browsers do not support video splitting.

The preprocessor performs GOP alignment for compatibility.

c) DAG generation

The preprocessor generates DAG pipelines based on configuration files written by developers.



A simplified DAG representation is shown in **Figure**.

```
task {
  name 'download-input'
  type 'Download'
  input {
    url config.url
  }
  output { it->
    context.inputVideo = it.file
  }
  next 'transcode'
}
```

```
task {
  name 'transcode'
  type 'Transcode'
  input {
    input context.inputVideo
    config config.transConfig
  }
  output { it->
    context.file = it.outputVideo
  }
}
```

Two code snippets are shown side-by-side. The left snippet defines a task named 'download-input' of type 'Download'. It has an input 'url' from 'config.url'. Its output is 'context.inputVideo = it.file'. It then has a 'next' statement pointing to 'transcode'. The right snippet defines a task named 'transcode' of type 'Transcode'. It has two inputs: 'input context.inputVideo' and 'config config.transConfig'. Its output is 'context.file = it.outputVideo'.

The configuration files used to generate DAG are shown in **Figure**.

Benefits:

- Flexible workflows
- Custom transcoding pipelines
- Parallel execution

d) Cache data

The preprocessor caches:

- GOP segments
- Metadata

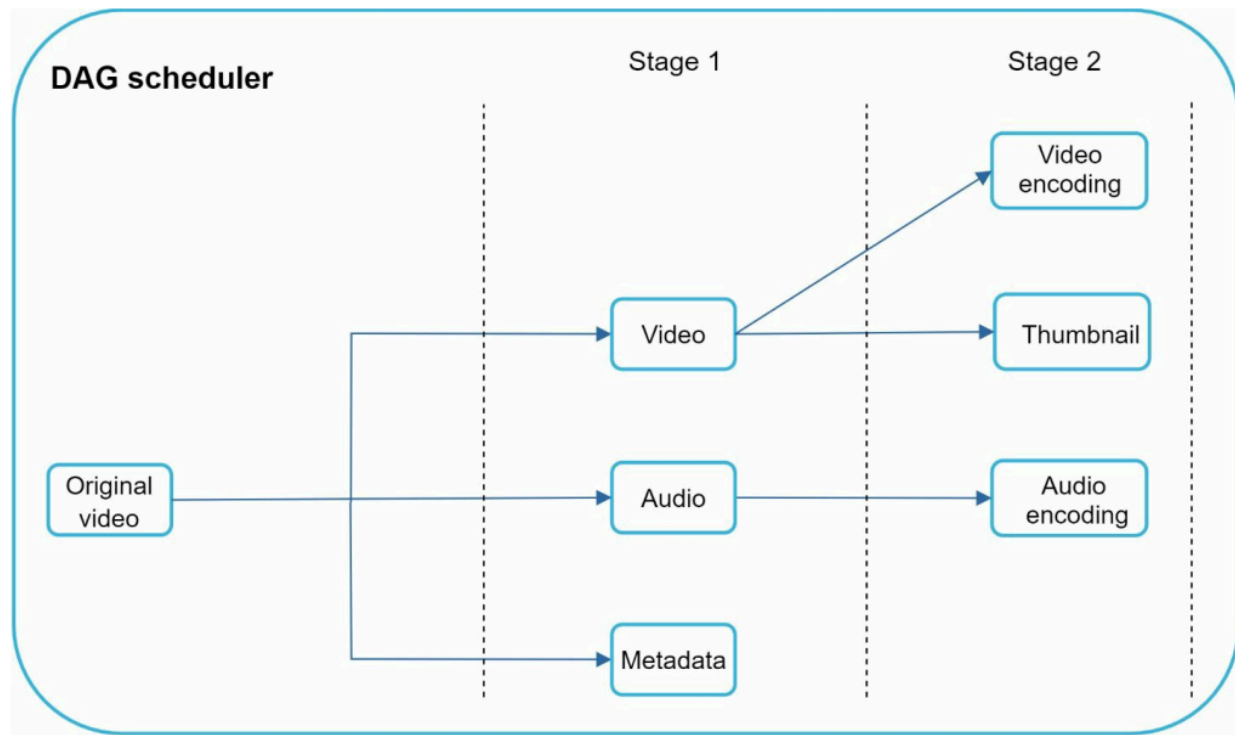
Benefits:

- Faster retries
- Better fault tolerance

If transcoding fails, cached segments are reused instead of re-uploading videos.

2. DAG Scheduler

The DAG scheduler splits the DAG into multiple stages and pushes tasks into task queues.



The working of DAG scheduler is shown in **Figure**.

Example stages:

Stage 1

- Extract video

- Extract audio
- Extract metadata

Stage 2

- Video encoding
- Audio encoding
- Thumbnail generation

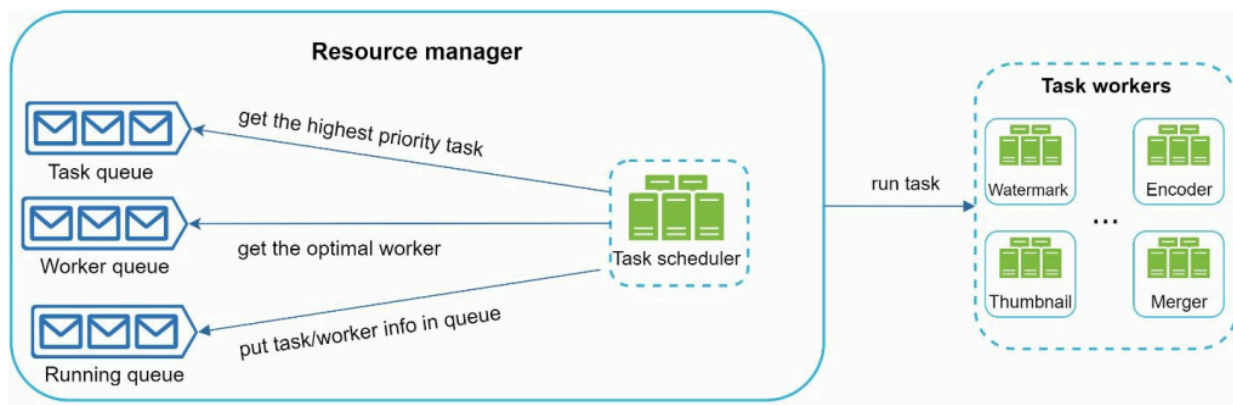
Independent tasks run in parallel.

Benefits:

- High throughput
- Better CPU utilization
- Faster transcoding

3. Resource Manager

The resource manager is responsible for efficient resource allocation.



Its architecture is shown in **Figure**.

The resource manager contains:

Task Queue

Stores pending transcoding tasks.

Usually implemented using:

- Priority queue

High-priority tasks are processed first.

Worker Queue

Stores:

- Worker availability
- Worker utilization
- Worker capabilities

Running Queue

Tracks:

- Currently running tasks
- Workers executing tasks

Task Scheduler

The task scheduler:

1. Picks highest priority task
2. Selects optimal worker
3. Assigns task
4. Updates running queue
5. Removes completed jobs

Benefits:

- Efficient load balancing

- Better resource utilization
- Reduced delay

4. Task Workers

Task workers execute transcoding jobs.

Different workers may specialize in:

- Video encoding
- Audio encoding
- Thumbnail generation
- Watermark processing

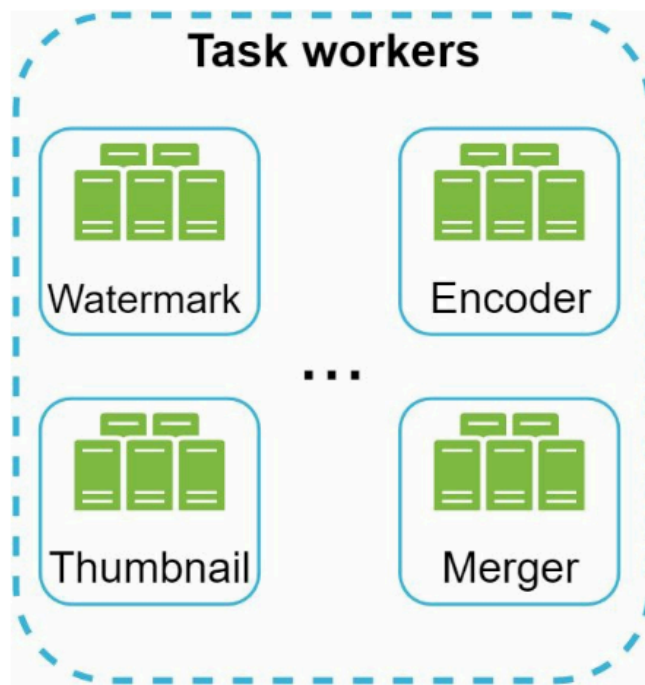


Figure shows different task workers running different tasks.

Workers can scale horizontally based on workload.

5. Temporary Storage

Multiple storage systems are used depending on data characteristics.

Metadata storage

Metadata is:

- Small in size
- Frequently accessed

Hence it is stored in:

- In-memory cache

Video/audio storage

Large binary files are stored in:

- Blob/Object storage

Examples:

- Amazon S3
- Google Cloud Storage

Cleanup

Temporary files are removed after transcoding completes.

Benefits:

- Reduced storage cost
- Efficient resource usage

6. Encoded Video Output

Encoded videos are the final outputs of the transcoding pipeline.

Examples:

- `funny_240p.mp4`
- `funny_720p.mp4`
- `funny_1080p.mp4`

These files are:

- Uploaded to CDN
- Distributed globally
- Streamed to users

Benefits of DAG-based transcoding architecture

Flexibility

Supports customized processing pipelines.

Parallelism

Independent tasks run simultaneously.

Fault tolerance

Failed tasks can retry independently.

Scalability

Task workers can scale horizontally.

Better resource utilization

Tasks are intelligently distributed across workers.

Overall transcoding workflow summary

1. User uploads video
2. Preprocessor splits video into GOP chunks
3. DAG pipeline is generated
4. DAG scheduler creates task stages
5. Resource manager assigns tasks to workers
6. Workers execute transcoding tasks
7. Encoded videos are generated
8. Files are stored in blob storage and CDN
9. Users stream optimized videos based on device and network conditions